

Notas API — de JSON a SQLAlchemy, explicado con tu propio código

Este documento repasa cada pieza del proyecto en el orden en que las fuiste construyendo, citando el código tal como está hoy en tus archivos.

1. El punto de partida: db.py (JSON a mano)

Antes de SQLAlchemy, la persistencia era leer y escribir un archivo `notas.json` completo en cada operación.

```
def cargar_db():
    if not DB_FILE.exists():
        return {"notas": []}
    contenido = DB_FILE.read_text(encoding="utf-8")
    return json.loads(contenido)

def guardar_db(datos: dict):
    contenido_json = json.dumps(datos, indent=4, ensure_ascii=False)
    DB_FILE.write_text(contenido_json, encoding="utf-8")
```

db.py

El problema de este enfoque no es que sea JSON — es que cada petición hace un **read-modify-write** no atómico sobre el archivo entero. Dos peticiones concurrentes pueden pisarse: A lee, B lee, A escribe, B escribe encima perdiendo el cambio de A. Esa es la *race condition* que motivó migrar a una base de datos real.

2. database.py — los modelos y el motor

2.1 Los modelos (las tablas)

```
class Usuario(Base):
    __tablename__ = "usuarios"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True)
    hashed_password: Mapped[str] = mapped_column(String(255))

class Nota(Base):
    __tablename__ = "notas"
    usuario_id: Mapped[int] = mapped_column(ForeignKey("usuarios.id"))
    id: Mapped[int] = mapped_column(primary_key=True)
    titulo: Mapped[str] = mapped_column()
    nota: Mapped[str] = mapped_column()
    creation_date: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    modification_date: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now(),
```

database.py

`Nota.usuario_id` con `ForeignKey("usuarios.id")` es la relación: cada nota pertenece a un único usuario. Es la columna que la Fase 3c va a usar para filtrar — hoy nadie la está usando en el GET, por eso `/notas` devuelve las notas de todos los usuarios mezcladas.

2.2 El motor y la activación de foreign keys

```
@event.listens_for(Engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()

engine = create_engine("sqlite:///notas.db", echo=True)
Base.metadata.create_all(bind=engine)
```

database.py

SQLite trae las foreign keys apagadas por defecto — sin este PRAGMA, ForeignKey("usuarios.id") sería solo decorativa, no bloquearía notas huérfanas. create_all crea las tablas si no existen (no las recrea si ya están).

2.3 get_session — la sesión por petición

```
def get_session() -> Generator[Session, None, None]:
    with Session(bind=engine) as session:
        yield session
```

database.py

El yield es la clave: la función se pausa aquí, FastAPI corre la ruta con esta session, y al terminar la ruta se reanuda — el with cierra la conexión solo. Una Session nueva por petición, nunca compartida entre peticiones concurrentes — es la misma clase de bug que el JSON, resuelto de raíz en vez de con cuidado manual.

3. main.py — cómo la ruta recibe esa sesión

```
SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()
```

main.py

Depends(get_session) le dice a FastAPI "antes de correr la ruta, ejecuta get_session y dame lo que produce". SessionDep es solo un alias para no repetir el trabalenguas Annotated[...] en cada ruta que necesite base de datos.

3.1 GET /notas — leer con SQLAlchemy

```
@app.get("/notas")
def notas_list(session: SessionDep) -> list[NotaOut]:
    notas = session.scalars(select(Nota)).all()
    return [NotaOut.model_validate(n) for n in notas]
```

main.py

select(Nota) arma la consulta, session.scalars(...).all() la ejecuta y trae los objetos Nota. Cada uno se convierte a NotaOut — no se devuelve el objeto SQLAlchemy directo.

3.2 NotaOut — el contrato de salida

```
class NotaOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    titulo: str
    nota: str
    usuario_id: int
    creation_date: datetime
    modification_date: datetime
```

main.py

Pydantic por defecto espera un diccionario para construirse. `from_attributes=True` le permite leer atributos de un objeto (`nota.titulo`) en vez de claves (`nota['titulo']`). El efecto práctico: el listado de campos aquí es el único lugar donde se decide qué sale al cliente — si mañana Usuario tuviera un campo sensible y por accidente terminara en una nota, `NotaOut` simplemente no lo dejaría pasar. Y como `-> list[NotaOut]` queda documentado, Swagger (`/docs`) sabe describir la forma exacta de la respuesta.

3.3 POST /notas — escribir con SQLAlchemy

```
class NotaIn(BaseModel):
    titulo: str
    nota: str

USUARIO_TEMPORAL = 1 # TODO: reemplazar por el usuario del token (Fase 3c)

@app.post("/notas", status_code=201, response_model=NotaOut)
def crear_nota(datos: NotaIn, session: SessionDep) -> Nota:
    nota = Nota(titulo=datos.titulo, nota=datos.nota, usuario_id=USUARIO_TEMPORAL)
    session.add(nota)
    session.commit()
    session.refresh(nota)
    return nota
```

main.py

`NotaIn` es el contrato de entrada (lo que el cliente puede mandar) — distinto de `NotaOut` (lo que recibe de vuelta). `session.refresh(nota)` es necesario porque tras el `commit`, SQLite ya le asignó un `id` y las fechas por `server_default` — sin `refresh`, ese objeto en memoria seguiría con esos campos vacíos.

`USUARIO_TEMPORAL` es el hueco explícito que deja pendiente la Fase 3c: hoy toda nota nueva se asigna al usuario 1 a mano.

3.4 DELETE y PATCH /notas/{titulo} — todavía en JSON

```
@app.delete("/notas/{titulo}")
def eliminar_nota(titulo: str) -> dict:
    db = cargar_db()
    total_antes = len(db["notas"])
    db["notas"] = [u for u in db["notas"] if u["titulo"] != titulo]
    if len(db["notas"]) < total_antes:
        guardar_db(db)
        return {"mensaje": "borrado con éxito!", "datos": titulo}
    else:
        raise HTTPException(status_code=404, detail="Elemento no encontrado")
```

main.py

Estas dos rutas son las últimas que quedan sobre `cargar_db()/guardar_db()` — el mismo patrón read-modify-write de la Sección 1, ahora conviviendo con las rutas ya migradas a SQLite. Migrarlas es el siguiente paso natural: mismo `select(Nota).where(Nota.titulo == titulo)` que ya usaste en tus pruebas de `database.py`, más `session.delete(nota)` o asignar campos y `session.commit()` para el PATCH.

4. El hueco que queda: identidad del usuario

Hoy GET /notas no filtra por usuario — devuelve todas las notas de todos los usuarios, y USUARIO_TEMPORAL = 1 en el POST asigna cualquier nota nueva al mismo usuario sin preguntar. La Fase 3c cierra esto: un token que el cliente manda, la ruta lo decodifica, y de ahí sale el `usuario_id` real para usar en `select(Nota).where(Nota.usuario_id == usuario_actual.id)` — la misma cláusula where que ya escribiste en tus pruebas manuales dentro de `database.py`, esta vez con un id que no viene hardcodeado.

5. Glosario — cada pieza, por dentro

Por cada término: qué es, dónde aparece en tu código, qué otra herramienta lo potencia, y cómo se integran entre sí.

FastAPI (la app)

Concepto: Framework web que traduce funciones Python en rutas HTTP, valida datos de entrada/salida y genera documentación automática.

```
app = FastAPI()

@app.get("/notas")
def notas_list(session: SessionDep) -> list[NotaOut]:
    ...
```

main.py

Se potencia con: Uvicorn, el servidor ASGI que efectivamente escucha el puerto y le pasa cada request a FastAPI (FastAPI define *qué* hacer, Uvicorn hace *que llegue*).

Cómo se integra: El decorador `@app.get("/notas")` registra la función como handler de ese verbo+ruta; FastAPI lee la firma (`session: SessionDep, -> list[NotaOut]`) para saber qué inyectar y qué validar.

Depends() — inyección de dependencias

Concepto: Mecanismo para que una función corra automáticamente antes de la ruta, prepare algo (como una conexión) y se lo entregue — sin que la ruta tenga que pedirlo a mano.

```
SessionDep = Annotated[Session, Depends(get_session)]

@app.get("/notas")
def notas_list(session: SessionDep) -> list[NotaOut]:
    ...
```

main.py

Se potencia con: Annotated (de typing), que es lo que permite adjuntarle metadata (Depends(...)) a un tipo (Session) sin inventar una clase nueva.

Cómo se integra: SessionDep empaqueta Session + Depends(get_session) en un solo alias. Cada ruta que lo usa como parámetro recibe una Session ya abierta, sin repetir with Session(bind=engine) en cada una.

Session (SQLAlchemy ORM)

Concepto: Objeto que representa una conversación con la base de datos: agrupa selects, inserts y commits en una unidad de trabajo.

```
def get_session() -> Generator[Session, None, None]:
    with Session(bind=engine) as session:
        yield session
```

database.py

Se potencia con: El generador de Python (yield): permite que get_session se "pause" mientras la ruta usa la session, y se reanuda (cerrando el with) cuando la ruta termina.

Cómo se integra: get_session() es la función que Depends() ejecuta. El with garantiza que la Session se cierre sola incluso si la ruta lanza una excepción — nunca queda una conexión abierta colgando.

select() y session.scalars()

Concepto: select(Nota) construye la consulta SQL ("SELECT * FROM notas") sin ejecutarla todavía; session.scalars(...) la ejecuta y devuelve los objetos Python resultantes.

```
notas = session.scalars(select(Nota)).all()
```

main.py

Se potencia con: El operador .where(), que se encadena sobre select() para filtrar — usado en tus pruebas de database.py como select(Nota).where(Nota.usuario_id == 1), y es exactamente lo que la Fase 3c necesita para no devolver notas de todos los usuarios.

Cómo se integra: select(Nota) sabe qué tabla y columnas consultar porque Nota es un modelo mapeado con DeclarativeBase — SQLAlchemy traduce la clase Python a SQL real.

DeclarativeBase / Mapped / mapped_column

Concepto: El sistema de SQLAlchemy para declarar tablas como clases Python: cada atributo Mapped[tipo] es una columna, con mapped_column() para configurarla (primary key, unique, foreign key...).

```
class Nota(Base):
    __tablename__ = "notas"
    usuario_id: Mapped[int] = mapped_column(ForeignKey("usuarios.id"))
    id: Mapped[int] = mapped_column(primary_key=True)
    titulo: Mapped[str] = mapped_column()
```

database.py

Se potencia con: Los type hints nativos de Python (int, str, datetime) — Mapped[int] no es magia, es el mismo sistema de tipos que usas en cualquier función, que SQLAlchemy lee para inferir el tipo de columna SQL.

Cómo se integra: class Nota(Base) hereda de Base (tu DeclarativeBase); esa herencia es lo que hace que Base.metadata.create_all(bind=engine) sepa qué tablas crear en SQLite.

ForeignKey

Concepto: Restricción que liga una columna a la clave primaria de otra tabla, garantizando que ese valor exista allá — evita notas "huérfanas" de un usuario inexistente.

```
usuario_id: Mapped[int] = mapped_column(ForeignKey("usuarios.id"))
```

database.py

Se potencia con: PRAGMA foreign_keys=ON, activado en el event.listens_for(Engine, "connect") — SQLite trae las foreign keys apagadas por defecto; sin ese PRAGMA, ForeignKey sería solo documentación, no una restricción real.

Cómo se integra: usuario_id en Nota apunta a usuarios.id. Es la columna que Fase 3c va a usar en el .where() para que cada usuario solo vea sus propias notas.

create_engine() y event.listens_for

Concepto: create_engine() abre el canal de conexión hacia el archivo notas.db (no conecta todavía, solo lo configura). event.listens_for engancha código a eventos del ciclo de vida del engine.

```
@event.listens_for(Engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()
```

```
engine = create_engine("sqlite:///notas.db", echo=True)
```

database.py

Se potencia con: echo=True, un flag del propio create_engine que imprime cada SQL ejecutado — la fuente de todas las líneas "sqlalchemy.engine.Engine ..." que ves en la consola al correr uvicorn.

Cómo se integra: El decorador se dispara cada vez que se abre una conexión física nueva a SQLite, así el PRAGMA queda activo para toda Session que use ese engine, sin tener que repetirlo a mano.

server_default / onupdate

Concepto: server_default=func.now() le dice a SQLite que ponga la fecha actual si no se especifica; onupdate=func.now() la actualiza sola en cada UPDATE de esa fila.

```
creation_date: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
modification_date: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

database.py

Se potencia con: `session.refresh(nota)`, usado en el POST — necesario porque estas fechas las calcula SQLite en el momento del commit, no Python; sin refresh, el objeto en memoria no las conocería.

Cómo se integra: `func.now()` (de SQLAlchemy) genera la función SQL nativa de fecha/hora — no es un `datetime.now()` de Python, es una instrucción que SQLite ejecuta al guardar.

BaseModel (Pydantic)

Concepto: Clase base para declarar la forma de los datos que entran o salen de una ruta: qué campos hay, de qué tipo, cuáles son opcionales.

```
class NotaIn(BaseModel):
    titulo: str
    nota: str

class NotaOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    titulo: str
    nota: str
    usuario_id: int
    creation_date: datetime
    modification_date: datetime
```

main.py

Se potencia con: Los type hints de Python otra vez (`str`, `int`, `datetime` | `None`) — Pydantic valida en tiempo de ejecución lo que el type checker solo revisa en el editor.

Cómo se integra: `NotaIn` valida lo que el cliente manda en el body del POST; `NotaOut` define lo que la ruta devuelve. Son dos contratos separados a propósito — entrada y salida no tienen por qué coincidir.

ConfigDict(from_attributes=True) y model_validate()

Concepto: `from_attributes=True` le permite a Pydantic leer un objeto por atributos (`nota.titulo`) en vez de solo por claves de diccionario (`nota['titulo']`). `model_validate()` dispara esa conversión.

```
notas = session.scalars(select(Nota)).all()
return [NotaOut.model_validate(n) for n in notas]
```

main.py

Se potencia con: Los objetos `Nota` que devuelve SQLAlchemy — sin `from_attributes`, Pydantic no sabría leerlos porque no son dicts.

Cómo se integra: Es el puente entre las dos librerías: SQLAlchemy entrega objetos ORM, Pydantic los traduce a algo serializable a JSON, y FastAPI se encarga de convertir eso a la respuesta HTTP final.

response_model / -> tipo de retorno

Concepto: Le dice a FastAPI qué forma debe tener la respuesta: valida los datos de salida y genera la documentación de Swagger.

```
@app.post("/notas", status_code=201, response_model=NotaOut)
def crear_nota(datos: NotaIn, session: SessionDep) -> Nota:
    ...
```

main.py

Se potencia con: Swagger UI (/docs), que lee response_model y la anotación -> para dibujar automáticamente el ejemplo de respuesta de cada ruta.

Cómo se integra: Aquí la ruta devuelve un objeto Nota (SQLAlchemy) pero response_model=NotaOut le indica a FastAPI que lo convierta a la salida de Pydantic antes de mandarlo — mismo truco que model_validate(), pero declarado en el decorador en vez de escrito a mano.

HTTPException

Concepto: Forma estándar de FastAPI para cortar la ejecución de una ruta y devolver un código de error HTTP con un mensaje.

```
if len(db["notas"]) < total_antes:
    guardar_db(db)
    return {"mensaje": " borrado con éxito!", "datos": titulo}
else:
    raise HTTPException(status_code=404, detail="Elemento no encontrado")
```

main.py

Se potencia con: Los códigos de estado HTTP como protocolo (404 = no encontrado, 201 = creado) — status_code=201 en el @app.post de crear_nota usa la misma idea del otro lado: éxito, no error.

Cómo se integra: raise HTTPException interrumpe la función igual que cualquier excepción de Python, pero FastAPI la intercepta y arma la respuesta JSON de error en vez de tumbar el servidor.

json (stdlib) — el sistema que se está reemplazando

Concepto: Módulo estándar de Python para convertir entre texto JSON y estructuras Python (dict, list).

```
def cargar_db():
    if not DB_FILE.exists():
        return {"notas": []}
    contenido = DB_FILE.read_text(encoding="utf-8")
    return json.loads(contenido)

def guardar_db(datos: dict):
    contenido_json = json.dumps(datos, indent=4, ensure_ascii=False)
    DB_FILE.write_text(contenido_json, encoding="utf-8")
```

db.py

Se potencia con: pathlib.Path, que da DB_FILE.exists() / .read_text() / .write_text() sin abrir/cerrar archivos a mano con open().

Cómo se integra: cargar_db() y guardar_db() todavía sostienen DELETE y PATCH /notas/{titulo} en main.py — son el código que migrar a select(Nota)/session.commit() dejaría obsoleto por completo.

6. Glosario técnico exhaustivo — estructura de 4 puntos

Por cada término: (1) definición general, (2) rol en este código, (3) integración y dependencias, (4) impacto en el flujo si faltara.

datetime

1. Definición general: Clase de la librería estándar de Python (`datetime.datetime`) que representa un instante en el tiempo (fecha + hora, opcionalmente con zona horaria). Es el tipo universal para timestamps en Python.

2. Rol en este código: En `main.py` (líneas 33-34) tipa los campos `creation_date` y `modification_date` del esquema `NotaOut`. En `database.py` (líneas 51-57) tipa las mismas columnas del modelo ORM `Nota` vía `Mapped[datetime]`.

3. Integración y dependencias: Es el puente de tipos entre las tres capas: SQLAlchemy guarda la columna como `DateTime(timezone=True)` en SQLite, la lee como objeto `datetime` de Python, y Pydantic (`NotaOut`) lo serializa automáticamente a string ISO 8601 en el JSON de respuesta (ej. "2026-07-15T10:30:00").

4. Impacto en el flujo: Los type hints fallarían con `NameError` al importar el módulo y la app no arrancaría. Sin fechas, se perdería la trazabilidad de cuándo se creó/modificó cada nota.

Annotated y Depends (inyección de dependencias)

1. Definición general: `Annotated` (de typing) permite adjuntar metadatos a un tipo: `Annotated[Session, Depends(get_session)]` significa "esto es una `Session`, y además, aquí está cómo obtenerla". `Depends` es el mecanismo de inyección de dependencias de FastAPI: en vez de que cada endpoint construya sus recursos, declara qué necesita y FastAPI se lo entrega ya resuelto.

2. Rol en este código: En `main.py` línea 12 se crea el alias `SessionDep = Annotated[Session, Depends(get_session)]`. Los endpoints `notas_list` (línea 38) y `crear_nota` (línea 53) declaran `session: SessionDep`, y FastAPI ejecuta `get_session()` en cada petición para inyectarles una sesión fresca.

3. Integración y dependencias: Conecta FastAPI con SQLAlchemy: `get_session` (`database.py` línea 69) es un generador que abre una `Session` ligada al engine, la cede con `yield`, y al terminar la petición el `with` la cierra automáticamente. El alias `SessionDep` evita repetir el `Annotated` completo en cada firma.

4. Impacto en el flujo: Sin `Depends`, FastAPI interpretaría `session` como un parámetro del body e intentaría parsearlo desde el JSON entrante → error 422 en cada llamada. Además cada endpoint tendría que abrir y cerrar sesiones a mano, con riesgo de fugas de conexiones.

FastAPI

1. Definición general: Framework web moderno de Python para construir APIs REST. Se apoya en type hints para validar datos automáticamente (vía Pydantic), generar documentación OpenAPI (`/docs`) y resolver dependencias. Corre sobre un servidor ASGI (uvicorn).

2. Rol en este código: `main.py` línea 13 crea la instancia `app = FastAPI()`. Es el objeto central: registra las 4 rutas, valida los bodies contra los modelos Pydantic, serializa las respuestas y expone la documentación interactiva.

3. Integración y dependencias: Sus decoradores (`@app.get`, etc.) registran funciones como endpoints. Lee los type hints de cada función para decidir qué viene del path (titulo: str), del body (datos: Notaln) o de una dependencia (session: SessionDep).

4. Impacto en el flujo: No hay aplicación. Sin app, uvicorn no tiene nada que servir.

HTTPException

1. Definición general: Excepción especial de FastAPI que, al lanzarse, se convierte en una respuesta HTTP con el código de estado y el detalle indicados, en vez de un error 500 genérico.

2. Rol en este código: En main.py líneas 77 y 106 devuelve un 404 Not Found con mensaje descriptivo cuando la nota a borrar o editar no existe.

3. Integración y dependencias: raise interrumpe la función inmediatamente; el manejador global de FastAPI la atrapa y produce {"detail": "Nota no encontrada"} con status 404. Es la forma idiomática de comunicar errores al cliente.

4. Impacto en el flujo: Si retornaras un dict de error, el cliente recibiría un 200 OK con mensaje de error dentro — semánticamente incorrecto y rompería a cualquier consumidor que confíe en los códigos HTTP. Sin manejar el caso, DELETE de una nota inexistente respondería "borrado con éxito" falsamente.

BaseModel y ConfigDict(from_attributes=True)

1. Definición general: BaseModel es la clase base de Pydantic: cualquier clase que herede de ella se convierte en un esquema con validación automática de tipos y serialización a/desde JSON. ConfigDict configura el comportamiento del modelo; from_attributes=True le permite construirse leyendo atributos de un objeto (obj.titulo) en vez de solo claves de un diccionario (dict['titulo']).

2. Rol en este código: Editarnota (línea 16), Crearnota (21), NotaOut (26) y Notaln (44) heredan de BaseModel. NotaOut lleva from_attributes=True en la línea 27 porque su fuente de datos son instancias ORM de Nota, que exponen sus campos como atributos, no como dict.

3. Integración y dependencias: Es EL puente Pydantic-SQLAlchemy. Sin from_attributes=True, NotaOut.model_validate(nota_orm) fallaría porque Pydantic esperaría un diccionario. Con él, lee nota_orm.id, nota_orm.titulo, etc. directamente.

4. Impacto en el flujo: Sin BaseModel no habría validación de entrada (un POST sin titulo pasaría) ni documentación de esquemas en /docs. Sin from_attributes=True, GET /notas y POST /notas lanzarían un ValidationError (error 500) al intentar convertir objetos ORM.

select

1. Definición general: Función de SQLAlchemy 2.x que construye una consulta SELECT de SQL de forma programática y segura (previene inyección SQL). select(Nota) genera SELECT * FROM notas.

2. Rol en este código: En main.py línea 40 construye la consulta para traer todas las filas de la tabla notas.

3. Integración y dependencias: select solo construye la consulta; no la ejecuta. Es la Session quien la ejecuta al pasársela: session.scalars(select(Nota)). Usa el modelo Nota de database.py como fuente de metadatos (sabe qué tabla y columnas consultar).

4. Impacto en el flujo: GET /notas no tendría forma de consultar la BD. La alternativa sería SQL crudo con strings, perdiendo seguridad y el mapeo automático a objetos.

Session y session.scalars().all()

1. Definición general: Session es la "unidad de trabajo" de SQLAlchemy: administra la conexión a la BD, rastrea los objetos que cargas o creas, y agrupa cambios en transacciones. .scalars() ejecuta una consulta y devuelve los objetos "planos" (instancias de Nota) en vez de tuplas Row; .all() los materializa en una lista.

2. Rol en este código: Se define en database.py línea 70 (dentro de get_session), se inyecta en los endpoints vía SessionDep, y en main.py línea 40 ejecuta el SELECT: session.scalars(select(Nota)).all() → list[Nota].

3. Integración y dependencias: La Session se liga al engine (bind=engine), que es quien conoce la URL sqlite:///notas.db. Sin .scalars(), session.execute(select(Nota)) devolvería filas tipo (Nota,) (tuplas de un elemento), incómodas de usar.

4. Impacto en el flujo: Sin Session no hay comunicación con la base de datos: ni lecturas ni escrituras. Sin .all(), tendrías un iterador perezoso que podría fallar al consumirse fuera del ciclo de vida de la sesión.

NotaOut.model_validate

1. Definición general: Método de clase de Pydantic v2 que valida y convierte un objeto arbitrario en una instancia del modelo. Es el reemplazo moderno de from_orm() de Pydantic v1.

2. Rol en este código: En main.py línea 41, dentro de la list comprehension, convierte cada instancia ORM Nota en un NotaOut serializable a JSON.

3. Integración y dependencias: Depende directamente de from_attributes=True para poder leer los atributos del objeto ORM. Actúa como filtro de salida: solo expone los campos declarados en NotaOut — si mañana Nota tuviera un campo secreto, no se filtraría al cliente.

4. Impacto en el flujo: Retornar los objetos Nota crudos con el hint -> list[NotaOut] también funcionaría (FastAPI hace la conversión implícitamente), pero la llamada explícita hace la conversión visible y falla temprano. Sin ninguna de las dos vías, el endpoint fallaría al serializar objetos SQLAlchemy, que no son JSON-serializables.

status_code=201 (en el decorador post)

1. Definición general: Parámetro del decorador de ruta que fija el código HTTP de éxito de la respuesta. 201 Created es el código semánticamente correcto cuando una petición crea un recurso nuevo (a diferencia del 200 OK por defecto).

2. Rol en este código: En main.py línea 52, hace que POST /notas responda 201 en vez de 200.

3. Integración y dependencias: FastAPI lo aplica automáticamente a toda respuesta exitosa del endpoint y lo documenta en OpenAPI/docs. No afecta a los errores (HTTPException lleva su propio código).

4. Impacto en el flujo: La app funcionaría igual, pero respondería 200 OK — semánticamente impreciso según la convención REST, y clientes o tests que verifiquen == 201 fallarían.

Métodos HTTP: @app.get, @app.post, @app.delete, @app.patch

1. Definición general: Decoradores que registran una función como manejador de un verbo HTTP + ruta. Los verbos expresan intención según REST: GET lee, POST crea, DELETE borra, PATCH modifica parcialmente (a diferencia de PUT, que reemplaza completo).

2. Rol en este código: @app.get("/notas") (línea 37) lista notas. @app.post("/notas") (línea 52) crea nota. @app.delete("/notas/{titulo}") (línea 65) borra por título; {titulo} es un path parameter que FastAPI extrae de la URL y pasa como argumento titulo: str. @app.patch("/notas/{titulo}") (línea 80) hace edición parcial combinando path param + body (Editarnota).

3. Integración y dependencias: Cada decorador conecta la URL con FastAPI, que enruta la petición, resuelve dependencias, valida el body y serializa la respuesta. La misma ruta /notas puede tener GET y POST porque el verbo distingue.

4. Impacto en el flujo: Sin el decorador, la función sería Python normal invisible para la web — la ruta devolvería 404/405. Usar el verbo equivocado (ej. GET para borrar) rompería la semántica REST: los GET deben ser seguros e idempotentes (cachés y crawlers podrían "borrar" notas).

El operador de unión de tipos: str | None = None

1. Definición general: str | None (sintaxis de Python 3.10+, equivalente a Optional[str]) declara que el valor puede ser un string o None. El = None le da valor por defecto, volviéndolo opcional.

2. Rol en este código: En Editarnota (main.py líneas 17-18). Es lo que hace posible el PATCH parcial: el cliente puede enviar solo {"titulo": "nuevo"} y cambios.nota quedará en None.

3. Integración y dependencias: Pydantic lo interpreta como "campo opcional" en la validación; el endpoint editar_notas lo explota con los chequeos if cambios.titulo is not None: (líneas 92, 96) para actualizar solo lo enviado.

4. Impacto en el flujo: Con titulo: str a secas, ambos campos serían obligatorios y un PATCH con un solo campo daría error 422 — el endpoint dejaría de ser una actualización parcial y se comportaría como un PUT.

session.add(), session.commit(), session.refresh()

1. Definición general: El trío del ciclo de escritura en SQLAlchemy. add() registra un objeto nuevo en la sesión (aún sin tocar la BD). commit() ejecuta el INSERT y confirma la transacción de forma permanente. refresh() recarga el objeto desde la BD para traer los valores que la base de datos generó, no Python.

2. Rol en este código: En main.py líneas 59-61, dentro de crear_notas. El refresh es crucial aquí: el id (autoincremental) y las fechas (server_default=func.now() en database.py) los genera SQLite, no Python. Sin recargar, el objeto en memoria tendría esos campos vacíos.

3. Integración y dependencias: Los tres operan sobre la Session inyectada por Depends. Después del refresh, el objeto Nota completo pasa por response_model=NotaOut para serializarse — que exige id, creation_date y modification_date presentes.

4. Impacto en el flujo: Sin add(), la sesión ignora el objeto y no hay INSERT. Sin commit(), la transacción se descarta al cerrar la sesión — la nota "se crea" pero desaparece. Sin refresh(), NotaOut fallaría al validar porque id y las fechas serían None → error 500.

Los dos flujos de persistencia: SQLAlchemy vs. db.py (cargar_db/guardar_db)

1. Definición general: El proyecto tiene dos backends de datos coexistiendo. (a) SQLAlchemy + SQLite (notas.db): base de datos real, con transacciones, tipos, claves foráneas y consultas SQL. (b) db.py: persistencia artesanal en un archivo notas.json — cargar_db() lee todo el JSON a un dict y guardar_db() reescribe el archivo completo.

2. Rol en este código: GET y POST (líneas 37-62) usan SQLAlchemy. DELETE y PATCH (líneas 65-110) usan el JSON. Esto refleja una migración a medias: los endpoints nuevos ya migraron a la BD; los viejos siguen en el archivo.

3. Integración y dependencias: No se integran — y ese es el bug latente. Son dos almacenes independientes: una nota creada con POST vive en notas.db y no puede borrarse con DELETE ni editarse con PATCH, porque esos buscan en notas.json. Diferencias clave: SQLite consulta selectivamente con SQL, tiene transacciones (commit/rollback), locking para concurrencia, integridad de tipos y FK, y campos autogenerados (id, fechas). El JSON carga TODO en memoria siempre, no tiene transacciones (un crash a mitad de write_text corrompe el archivo), dos peticiones simultáneas se pisan (última escritura gana), y no tiene ninguna integridad ni campos autogenerados.

4. Impacto en el flujo: Si eliminaras db.py, DELETE y PATCH morirían con ImportError al arrancar. Pero el impacto real es funcional: la API es inconsistente hoy — crear y luego borrar la misma nota devuelve 404. El siguiente paso natural del proyecto es migrar DELETE y PATCH a SQLAlchemy y borrar db.py.

DeclarativeBase, Mapped y mapped_column

1. Definición general: El sistema declarativo de SQLAlchemy 2.x. DeclarativeBase es la clase raíz de la que heredan los modelos; Mapped[tipo] declara el tipo Python de una columna; mapped_column() define sus propiedades SQL (primary key, defaults, constraints).

2. Rol en este código: Base (database.py línea 26) es la raíz; Usuario y Nota la heredan y así SQLAlchemy sabe que representan las tablas usuarios y notas. Mapped[int] = mapped_column(primary_key=True) deduce el tipo SQL (INTEGER) del hint Python.

3. Integración y dependencias: Cada clase que hereda de Base se registra en Base.metadata, que create_all usa para generar el CREATE TABLE. Los hints Mapped[datetime] son los mismos tipos que Pydantic espera en NotaOut — la coherencia de tipos entre capas es lo que hace que model_validate funcione sin conversiones manuales.

4. Impacto en el flujo: Sin Base, no hay registro de tablas ni ORM: select(Nota) no sabría a qué tabla apunta y create_all no crearía nada.

ForeignKey('usuarios.id')

1. Definición general: Restricción de integridad referencial: el valor de esta columna debe existir como id en la tabla usuarios.

2. Rol en este código: Nota.usuario_id (database.py línea 43) liga cada nota a su dueño. Es la base del multi-usuario que el TODO de main.py línea 49 anticipa (hoy todas las notas se crean con USUARIO_TEMPORAL = 1).

3. Integración y dependencias: Solo funciona gracias al listener del PRAGMA, porque SQLite ignora las FK por defecto. Si el usuario 1 no existe en la tabla usuarios, el commit() de crear_nota lanzará IntegrityError.

4. Impacto en el flujo: Podrían existir notas huérfanas apuntando a usuarios inexistentes — corrupción lógica silenciosa.

@event.listens_for(Engine, 'connect') + PRAGMA foreign_keys=ON

1. Definición general: El sistema de eventos de SQLAlchemy permite ejecutar código en momentos del ciclo de vida (aquí: cada nueva conexión). El PRAGMA es una peculiaridad histórica de SQLite: soporta claves foráneas pero las trae desactivadas por compatibilidad, y hay que activarlas por conexión.

2. Rol en este código: En database.py líneas 13-20 intercepta cada conexión que el engine abre y ejecuta el PRAGMA antes de que se use.

3. Integración y dependencias: Es el habilitador real del ForeignKey. Sin él, la restricción existe en el schema pero SQLite no la aplica.

4. Impacto en el flujo: Ningún error visible — las FK simplemente dejarían de validarse. Es el tipo de fallo silencioso más peligroso.

create_engine y Base.metadata.create_all

1. Definición general: create_engine crea el objeto que administra las conexiones a la BD a partir de una URL (sqlite:///notas.db = archivo local). create_all emite CREATE TABLE IF NOT EXISTS para cada modelo registrado. echo=True imprime todo el SQL ejecutado (modo aprendizaje/debug).

2. Rol en este código: En database.py líneas 62-64 se ejecutan al importar el módulo: la BD y sus tablas existen antes de la primera petición.

3. Integración y dependencias: El engine es lo que get_session liga a cada Session (bind=engine); toda operación de los endpoints GET/POST termina pasando por él.

4. Impacto en el flujo: Sin engine, no hay conexión posible. Sin create_all, la primera consulta fallaría con "no such table: notas" en una BD nueva. (Nota: create_all no migra tablas existentes — si cambias un modelo, no altera la tabla ya creada; para eso se usa Alembic.)

get_session con yield (Generator)

1. Definición general: Una dependencia con yield en FastAPI tiene dos fases: lo anterior al yield corre antes del endpoint (setup), y lo posterior — aquí, la salida del with, que cierra la sesión — corre después de enviada la respuesta (teardown). Patrón clásico de gestión de recursos por petición.

2. Rol en este código: En database.py líneas 69-71 garantiza una sesión nueva por petición y su cierre garantizado, incluso si el endpoint lanza una excepción.

3. Integración y dependencias: Es la función que Depends(get_session) invoca; el with Session(bind=engine) la conecta al engine. El tipo de retorno Generator[Session, None, None] documenta que cede una Session.

4. Impacto en el flujo: Si devolviera la sesión con return en vez de yield, nadie la cerraría → fuga de conexiones. Si fuera una sesión global compartida, dos peticiones concurrentes corromperían el estado

transaccional mutuamente.

func.now() con server_default y onupdate

- 1. Definición general:** func.now() genera la expresión SQL now()/CURRENT_TIMESTAMP: la fecha la pone el motor de BD, no Python. server_default la aplica al insertar; onupdate la reaplica en cada UPDATE.
- 2. Rol en este código:** En database.py líneas 52-56 automatiza creation_date y modification_date de Nota sin que ningún endpoint tenga que tocarlas.
- 3. Integración y dependencias:** Es la razón de que exista session.refresh() en crear_notas: los valores nacen en la BD y hay que traerlos de vuelta al objeto Python para que NotaOut los serialice.
- 4. Impacto en el flujo:** Los campos serían NULL al insertar → NotaOut fallaría la validación (espera datetime, no None) → error 500 en el POST.

response_model=NotaOut

- 1. Definición general:** Parámetro del decorador que declara el esquema de la respuesta. FastAPI valida y filtra lo que retorna la función contra ese modelo antes de enviarlo.
- 2. Rol en este código:** En main.py línea 52, crear_notas retorna un objeto ORM Nota crudo; response_model=NotaOut lo convierte automáticamente (equivale a un model_validate implícito) y documenta el schema de salida en /docs.
- 3. Integración y dependencias:** Depende de from_attributes=True en NotaOut para leer el objeto ORM. Contrasta con notas_list, que hace la conversión explícita — son las dos formas válidas del mismo patrón.
- 4. Impacto en el flujo:** FastAPI intentaría serializar el objeto SQLAlchemy directamente y fallaría (los objetos ORM no son JSON-serializables), o expondría campos internos si algún día el modelo crece.

Resumen arquitectónico

FastAPI recibe y valida con Pydantic (NotaIn/Editarnotas en la entrada, NotaOut en la salida), la inyección de dependencias entrega una Session por petición, SQLAlchemy traduce objetos a SQL sobre SQLite — y queda pendiente migrar DELETE/PATCH del viejo almacén JSON (db.py) a la base de datos, porque hoy los dos flujos no se ven entre sí.